

Universidade Federal Do Mato Grosso Do Sul

Curso Superior De Tecnologia Em Inteligência Artificial

Algoritmos 1 — Professor: Aumary Junior

Aluno: Saulo Ferro Maciel

Avaliação do Módulo 3

1) INVESTIGAÇÃO TÉCNICA

Nesta etapa, você deve analisar o código sem o uso do computador inicialmente, e depois validar suas descobertas.

- Olhando para os "Dados de Entrada para Teste", qual será o valor exato impresso para árvores, obstáculos e linha_maior? Justifique como o laço aninhado chega a esses números.
- Execute o programa na sua IDE com os dados fornecidos. Sua previsão estava correta? Se houve erro, identifique em qual ponto da lógica você se equivocou.
- Por fim, pesquise e responda as questões abaixo:
 - o Qual a função da variável `cont` e por que ela precisa ser zerada a cada iteração do primeiro laço `for`?
 - o Explique a diferença técnica entre acessar `mapa[i]` e `mapa[i][j]`.
 - o Quais estruturas de dados compostas foram utilizadas e por que elas são adequadas para este problema?.

RESPOSTA = Achei que sairia 6 árvores, 6 obstáculos e 12 caminhos livres, porém achei que ele iria apontar como a primeira linha como a maior com mais árvores. Mas, depois de executar, e ver que o código funciona melhor com maior número de linha do que colunas, conseguindo contar assim devido ao “for aninhado”. Se colocar ao contrário (mais colunas do que linhas), o código se perde na contagem.

No final, o “`cont`” contabiliza as árvores numa linha, se tiver uma árvore, muda de zero para um. Depois de executar o `for` aninhado, ele sempre checa se `cont` é maior que zero, e como o loop aninhado faz por linha, ele registra a quantidade de árvores numa linha em “maior” que passa a posição da linha com mais árvores para `linha_maior`, e zera o `cont`, e volta a contar uma nova linha, no entanto, só irá registrar na variável maior se a próxima linha tiver mais árvores que a anterior.

Acessar `mapa[i]` significa que só há uma dimensão, logo, um vetor simples, agora com dois índices seria uma matriz bidimensional. Estamos usando listas, matrizes de listas, para assim armazenar uma conjuntura de dados por cada posição da matriz.

2) DESAFIO DE MODIFICAÇÃO E CRIAÇÃO DE FUNCIONALIDADE

Agora, você deve aplicar o que aprendeu para expandir o sistema.

- Altere o código para que ele também informe o percentual do mapa ocupado por árvores e exiba uma classificação:
 - o Menor que 20%: "Pouco arborizado"
 - o Entre 20% e 50%: "Arborização média"
 - o Acima de 50%: "Muito arborizado"
- Desenvolva uma nova funcionalidade que identifique e imprima as coordenadas (linha, coluna) de todas as árvores que estão localizadas apenas nas bordas do mapa (primeira/última linha ou primeira/última coluna). Se não houver árvores nas bordas, exiba: "Nenhuma árvore na borda".
-

RESPOSTA = Movi as variáveis importantes para o início do código, as tornando-as globais. Segundo, dentro do 1º loop for, criei um registro bidimensional das coordenadas e salva a linha e coluna.

No 2º loop for, depois do for aninhado e da atribuição na variável "maior", criei a variável porcentagem que visa pegar cont multiplicar com 100 e divide pelas colunas, assim obtêm-se o percentual, no caso, a variável que declarei percentual guarda o percentual mais uma string informativa.

A ultima estrutura que fiz foi um loop for para imprimir corretamente a soma as coordenadas das árvores.

```
home > saulo > Área de trabalho > prova3.py > ...
1  # PRINCIPAIS VARIÁVEIS GLOBAIS
2  linhas = int(input("Qtd de linhas: "))
3  colunas = int(input("Qtd de colunas: "))
4  arvores = obstaculos = livres = 0
5  maior = linha_maior = percentual = -1
6  porcentagem, cordenadas, mapa = 0, [], []
7
8  # LOOP FOR QUE ALEM DE CAPTURAR AS LINHAS, VERIFICA SE EXISTE ARVORE NAS EXTREMIDADES
9  # REGISTRA AS COORDENADAS DAS ARVORES NAS EXTREMIDADES EM UMA LISTA
10 for i in range(linhas):
11     linha = list(input(f"Digite a linha {i+1}: "))
12     if linha[0] == "A":
13         arvore_borda = [f"Linha {i+1}", f"Coluna 0"]
14         cordenadas.append(arvore_borda)
15     if linha[-1] == "A":
16         arvore_borda = [f"Linha {i+1}", f"Coluna {colunas-1}"]
17         cordenadas.append(arvore_borda)
18     mapa.append(linha)
19
20 for i in range(linhas):
21     cont = 0
22     for i in range(colunas):
```

```

for i in range(linhas):
    cont = 0
    for j in range(colunas):
        if mapa[i][j] == 'A':
            arvores += 1
            cont += 1
        elif mapa[i][j] == '#': obstaculos += 1
        else: livres += 1

    if cont > maior:
        maior = cont
        linha_maior = i

# CALCULO DO PERCENTUAL DE ARVORES E CLASSIFICAÇÃO DE ARBORIZAÇÃO
porcentagem = (cont*100)/colunas
if porcentagem < 20: percentual = f"{porcentagem:.0f}% - pouco arborizado"
elif porcentagem >= 20 and porcentagem < 50: percentual = f"{porcentagem:.0f}% - Arborização média"
else: percentual = f"{porcentagem:.0f}% - Muito arborizado"

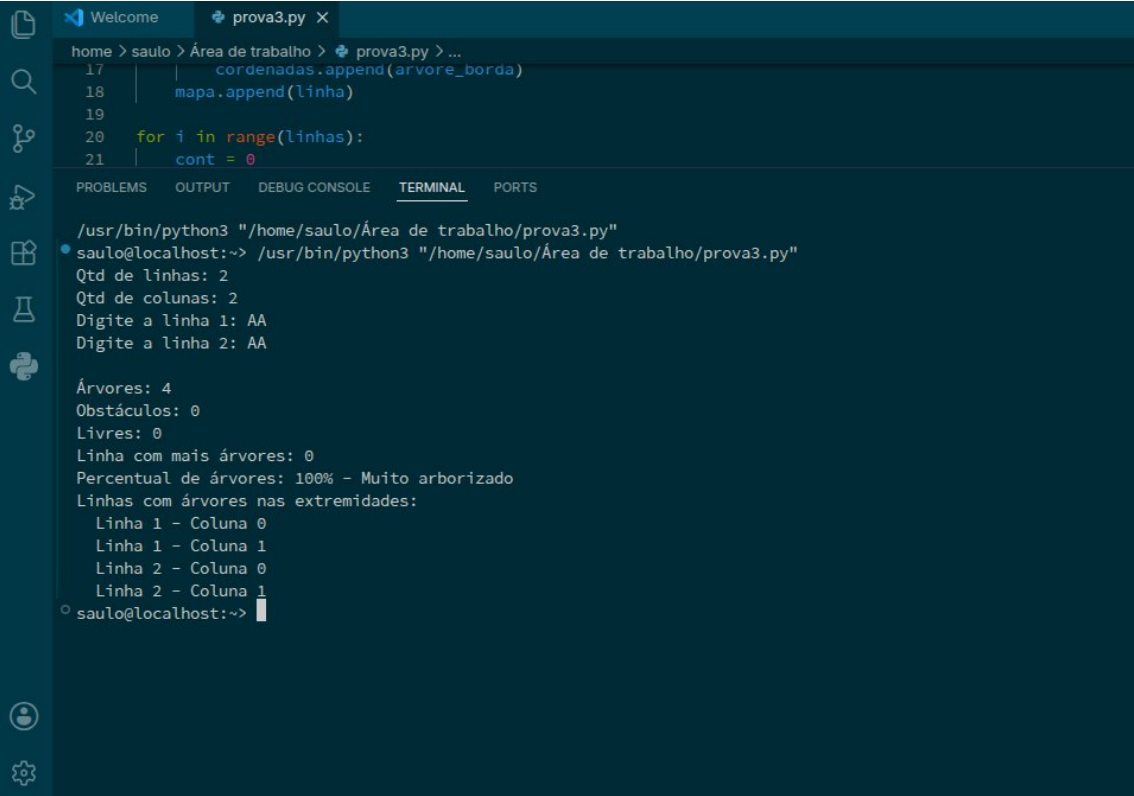
```

home > saulo > Área de trabalho > prova3.py > ...

```

17 | cordenadas.append(arvore_borda)
18 | mapa.append(linha)
19 |
20 | for i in range(linhas):
21 |     cont = 0
22 |     for j in range(colunas):
23 |         if mapa[i][j] == 'A':
24 |             arvores += 1
25 |             cont += 1
26 |         elif mapa[i][j] == '#': obstaculos += 1
27 |         else: livres += 1
28 |
29 |     if cont > maior:
30 |         maior = cont
31 |         linha_maior = i
32 |
33 | # CALCULO DO PERCENTUAL DE ARVORES E CLASSIFICAÇÃO DE ARBORIZAÇÃO
34 | porcentagem = (cont*100)/colunas
35 | if porcentagem < 20: percentual = f"{porcentagem:.0f}% - pouco arborizado"
36 | elif porcentagem >= 20 and porcentagem < 50: percentual = f"{porcentagem:.0f}% - Arborização méd
37 | else: percentual = f"{porcentagem:.0f}% - Muito arborizado"
38 |
39 | print(f"\nÁrvores: {arvores}")
40 | print(f"Obstáculos: {obstaculos}")
41 | print(f"Livres: {livres}")
42 | print(f"Linha com mais árvores: {linha_maior}")
43 | print(f"Percentual de árvores: {percentual}")
44 |
45 | # IMPRESSÃO DAS COORDENADAS DAS ARVORES NAS EXTREMIDADES
46 | print("Linhas com árvores nas extremidades:")
47 | for coordenada in cordenadas:
48 |     print(f" {coordenada[0]} - {coordenada[1]}")

```



The image shows a Visual Studio Code editor window with a file named `prova3.py` open. The editor has a dark theme. The file content is as follows:

```
17 |         coordenadas.append(arvore_borda)
18 |     mapa.append(linha)
19 |
20 |     for i in range(linhas):
21 |         cont = 0
```

Below the editor, the **TERMINAL** tab is active, showing the execution of the script. The output is as follows:

```
/usr/bin/python3 "/home/saulo/Área de trabalho/prova3.py"
saulo@localhost:~$ /usr/bin/python3 "/home/saulo/Área de trabalho/prova3.py"
Qtd de linhas: 2
Qtd de colunas: 2
Digite a linha 1: AA
Digite a linha 2: AA

Árvores: 4
Obstáculos: 0
Livres: 0
Linha com mais árvores: 0
Percentual de árvores: 100% - Muito arborizado
Linhas com árvores nas extremidades:
  Linha 1 - Coluna 0
  Linha 1 - Coluna 1
  Linha 2 - Coluna 0
  Linha 2 - Coluna 1
saulo@localhost:~$
```